

Supplementary Material for “Extraction meets Distillation”

Biprarshi Biswas¹, Arya Anshuman Ojha², Yubraj Das¹, Devansh Khosla³,
Kamran Rais³, Saankhya Samanta⁴, and Subhamoy Maitra⁵

¹ Jadavpur University, Kolkata, India

{biprarshib, yyubarajddas9876}@gmail.com

² National Institute of Science Education and Research, Bhubaneswar, India

kalingasandhababun@gmail.com

³ Institute of Mathematics and Application, Bhubaneswar, India

{devanshkhosla289, kamranrais41}@gmail.com

⁴ Indian Institute of Technology, Kharagpur, India

saankhya.samanta99@gmail.com

⁵ Indian Statistical Institute, Kolkata, India

{subho@isical.ac.in, maitra.subhamoy@gmail.com}

This document is the supplementary material for CSCML 2026 submission 183, *Extraction meets Distillation — Cryptanalytic techniques to recover Parameters from a Deep Neural Network*. Section 1 records the background material and notation used throughout the paper, for completeness. Sections 2–7 then collect the supporting experimental detail deferred from the main body: the dataset contract and the reference attack parameters, the ReLU versus Leaky ReLU scaling comparison, the A/B of the two sign optimisers, the per-metric EQS breakdown for the two CIFAR-10 flagships, and the additive Phase-3 ablation. The main paper’s Table 1 (`tab:permodel`, per-victim extraction results) and Table 2 (`tab:eqs`, composite EQS for the extraction and distillation arms) report the headline numbers that the tables here expand on; the five places in the main body that defer to “the supplementary material” all point to this document. Nothing here is new — every definition, number, caption and table is reproduced unchanged from the paper’s original appendices.

1 Additional Background Materials

Here we add background materials for completeness. We use popular notation throughout, if not otherwise mentioned.

1.1 Deep Neural Networks (DNNs)

We study fully connected feedforward networks $f_\theta : \mathbb{R}^{d_0} \rightarrow \mathbb{R}^{d_{r+1}}$ composed of $r + 1$ affine layers separated by an element wise nonlinearity σ :

$$f_\theta = f_{r+1} \circ \sigma \circ f_r \circ \cdots \circ \sigma \circ f_1, \quad f_i(\mathbf{x}) = A^{(i)}\mathbf{x} + \mathbf{b}^{(i)}, \quad (1)$$

where $A^{(i)} \in \mathbb{R}^{d_i \times d_{i-1}}$ and $\mathbf{b}^{(i)} \in \mathbb{R}^{d_i}$ represent the weight matrix and bias vector of layer i , whose individual rows $A_j^{(i)}$ and entries $b_j^{(i)}$ define the weight vector and

bias of neuron (i, j) respectively. The network contains a total of $N = \sum_{k=1}^r (d_k)$ hidden neurons. The final layer (f_{r+1}) output is passed to a softmax function to yield a class probability vector, from which the arg max operator selects the index of the highest probability to produce the final class label. The complete trainable parameter set is defined as:

$$\theta = \left\{ \left(A^{(i)}, b^{(i)} \right) \right\}_{i=1}^{r+1}. \quad (2)$$

1.2 Critical Hyperplanes and Decision Boundaries

A region boundary at which a single neuron (i, j) switches sign while all other neurons keep theirs is the *critical hyperplane* of that neuron

$$\mathcal{H}_{i,j} = \{ \mathbf{x} \in \mathbb{R}^{d_0} : \eta_{i,j}(\mathbf{x}) = 0 \}.$$

where $\eta_{i,j}(\mathbf{x})$ represents the pre-activation value of a neuron (i, j) . For a first-layer neuron $\mathcal{H}_{1,j}$ is a hyperplane in the input space with normal $A_j^{(1)}$. For a deeper neuron it is a piecewise-linear surface whose local orientation is given by an effective normal $\tilde{A}_j^{(i)}$. The oracle separately partitions the input space by predicted label. The *decision boundary* between classes m and n is

$$\mathcal{D}_{m,n} = \{ \mathbf{x} \in \mathbb{R}^{d_0} : [f_\theta(\mathbf{x})]_m = [f_\theta(\mathbf{x})]_n = \max_k [f_\theta(\mathbf{x})]_k \}.$$

Decision boundaries are themselves piecewise-linear and they bend wherever they cross a critical hyperplane, since the underlying affine map $M_\rho x + c_\rho$ changes across the crossing and the boundary changes orientation with it.

1.3 Dual Points and Dual Spaces

The central primitive the pipeline exploits is a point that is critical for the oracle. A point x^* is a *dual point* for neuron (i, j) if it lies on that neuron's critical hyperplane and simultaneously on some decision boundary, that is $x^* \in \mathcal{H}_{i,j} \cap \mathcal{D}_{m,n}$ for some $m \neq n$. Its *dual space* is the local intersection $\mathcal{S}_{x^*} = \mathcal{H}_{i,j} \cap \mathcal{D}_{m,n}$ in a neighbourhood of x^* . The attacker never observes $\mathcal{H}_{i,j}$ directly and works only from labels, recovering each dual point from the bend

1.4 Activation Functions

We consider two piecewise-linear (PWL) activation functions: the Rectified Linear Unit, $\text{ReLU}(t) = \max(0, t)$, and its generalization, Leaky ReLU, defined for a fixed slope $\alpha > 0$ (typically $\alpha \in [10^{-3}, 10^{-1}]$) [1] as:

$$\text{ReLU}(t) = \max(0, t) \quad \text{and} \quad \text{LReLU}_\alpha(t) = \max(\alpha t, t) = \begin{cases} t & t > 0, \\ \alpha t & t \leq 0. \end{cases} \quad (3)$$

While ReLU completely zeroes out negative pre-activations so that an inactive neuron contributes nothing downstream, LReLU $_{\alpha}$ allows inactive neurons to pass an α -scaled leakage signal. Dynamically, ReLU can be viewed as the limiting case where $\alpha = 0$. Crucially, the zero-set of the pre-activation is entirely independent of α , allowing our geometric attack to uniformly target both activation topologies.

1.5 Multinomial Logistic Regression

Logistic Regression, a popular Machine Learning Algorithm was designed to handle binary classification. To extend it to the multinomial classification domain, Multinomial Logistic Regression (or also Softmax Regression) was introduced. Unlike Logistic Regression which uses a sigmoid activation function, multinomial logistic regression uses the softmax activation function to model the probabilities of multiple classes simultaneously.

Sigmoid Function Softmax Function

$$\sigma(z) = \frac{1}{1+e^{-z}}, \quad \text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}},$$

where K is the number of classes, and $\text{Softmax}(z_i)$ is the probability of being in the class i .

1.6 Gradient Descent

Gradient Descent is a popular optimisation technique especially to minimise functions where obtaining a closed form equation is difficult (i.e., finding their optima by equating their 1st derivative to 0). In such a case the objective function (say J) is minimised in an iterative manner. The process starts with an initialisation of parameters (say θ) to be optimised, and their values are updated iteratively using the gradient of the objective function as:

$$\theta := \theta - \eta \frac{\partial J(\theta)}{\partial \theta}$$

where, η is a hyperparameter called the learning rate and decides how much the gradient will affect the value of the parameter. This process repeats till the value of the parameters converge.

1.7 Activation Patterns and Affine Regions

For a fixed network f_{θ} , each input x induces an **activation pattern** $\rho(x) \in \{0, 1\}^N$ defined element-wise over all hidden neurons by the indicator function:

$$\rho_{i,j}(x) = \mathbf{1} [\eta_{i,j}(x) > 0]. \quad (4)$$

Inputs sharing an identical activation pattern ρ restrict the network to a single, locally affine mapping $f_{\theta}|_{\rho}(x) = M_{\rho}x + c_{\rho}$. The closure of each such input set

forms an **affine region**. Collectively, these regions form a piecewise-affine partition of the input space, meaning f_θ is locally linear within each region, while its non-linear characteristics are strictly confined to the regional boundaries.

2 Experimental setup tables

Evaluation methodology. Every victim is extracted under one three-tier dataset contract (Table 1). Two oracle- slices form the Phase-3 training pool and a third disjoint slice is held out for every reported number, so training and evaluation inputs never overlap.

Table 1. Dataset contract, uniform across all victims

Slice	Role	Source (<code>make_blobs</code> / CIFAR-10)
X_{test}	Phase-3 training pool	seed 42 sample / test batch
$X_{\text{test}2}$	Phase-3 training pool	seed 99 sample / train idx 40000–49999
$X_{\text{test}3}$	Held-out evaluation + watchdog	seed 123 sample / train idx 10000–19999

3 Reference Attack Parameters

This subsection records the exact parameters used for every experiment reported in this work, for reproducibility. These values were chosen for experimentation under our compute budget and carry no deeper justification. We did not tune them toward a theoretical optimum, and different settings would likely trade run time against recovery quality without changing the qualitative results.

Table 2 lists the per-architecture settings that vary across the victim suite. Table 3 lists the pipeline parameters that are held fixed across all victims. The `make_blobs` values are hardcoded in the batch driver of the runs. The CIFAR-10 flagship uses a separate driver retuned to the values in the `full` column, run on a single V100 GPU in float64.

Table 2. Per-architecture tuning. Each dual-search round emits the per-architecture target triplet count (`tiniest` 3000, `tinier` 2000, `tiny` and `full` 10000). PT+margin is intractable on the 256-wide CIFAR architecture, so the flagship uses SA+margin only.

Parameter	<code>tiniest</code>	<code>tinier</code>	<code>tiny</code>	<code>full</code> (CIFAR)
dual-search rounds	6	8	20	150
dual workers / batch	7 / 256	7 / 256	7 / 256	1–3 / 256
target triplets / round	3000	2000	10000	10000
sign restarts	1	1	2	0–1
sign pair-lookahead	8	8	8	4
sign refine cycles	3	3	3	1
refine epochs	300	500	500	500

Table 3. Pipeline parameters held fixed across all victims. Both activations share these settings; only **LEAKY_ALPHA** distinguishes the ReLU and Leaky ReLU runs.

Phase	Parameter	Value
Dual search	implementation	<code>torch</code> , CUDA float64
Sign recovery	runner	<code>batched_sign_recovery.py</code> , float64
Reconstruction	sign-search method	SA+margin (default); PT+margin (A/B arm)
	mini-refine	20 epochs, lr 5×10^{-3} , wd 10^{-4}
	refine optimiser	AdamW, cosine lr, wd 10^{-4}
	watchdog	early stop, patience 5, eval every 10
	eval gating	eval on X_{test3} , train on $X_{\text{test}} \cup X_{\text{test2}}$ (20K)
Activation	LEAKY_ALPHA	0.0 (ReLU) / 0.01 (Leaky ReLU)
Oracle cost	batched argmax queries	3 per model (sign search, fc5 fit, refinement)

4 ReLU versus Leaky ReLU scaling

Table 4 upholds the effect of the activation on Phase-1-2 recovery. The advantage of Leaky ReLU increases as the architecture widens, therefore the gap between the two recovered-count columns is high for the larger architectures. The two fc4 columns show where this advantage originates, in the deepest hidden layer that ReLU leaves almost unrecovered. The effect is structural rather than incidental, because it strengthens with depth and width together.

Table 4. Phase-1 recovery, ReLU vs Leaky ReLU. The Leaky advantage grows with scale (+5 / +6 / +87 / +308 neurons)

Architecture	ReLU recovered	Leaky recovered	Delta	fc4 ReLU	fc4 Leaky
tiniest	20/32	25/32	+5	5/8	6/8
tinier	29/56	35/56	+6	0/8	2/8
tiny	139/256	226/256	+87	0/64	54/64
cifar	499/832	807/832	+308	0/64	58/64

5 Simulated Annealing Versus Parallel Tempering

Table 5 holds the recovered signatures fixed and varies only the sign optimizer, so any difference is attributable to the search and not to the upstream recovery. The two optimizers return the same assignment on every saturated victim. They diverge on the single unsaturated victim, the tiny ReLU row, where parallel tempering improves on simulated annealing. That row is the only place where the choice of optimizer changes the outcome.

Table 5. A/B of the per-layer sign optimiser on identical recovered signatures. SA and PT are identical on 5/6 victims (saturation watchdog); PT wins only on the unsaturated tiny_relu.

Model	SA+margin (sign-acc / EQS)	PT+margin (sign-acc / EQS)	Delta
tiniest_relu	0.601 / 69.6	0.601 / 69.6	identical
tiniest_leakyrelu	0.771 / 83.9	0.771 / 83.9	identical
tinier_relu	0.347 / 76.7	0.347 / 76.7	identical
tinier_leakyrelu	0.463 / 81.0	0.463 / 81.0	identical
tiny_relu	0.503 / 75.2	0.518 / 77.0	PT +1.5pt sign-acc, +1.8 EQS
tiny_leakyrelu	0.500 / 73.2	0.500 / 73.2	identical

6 CIFAR-10 EQS detail

Tables 6 and 7 give the per-metric EQS breakdown for the two CIFAR flagships, expanding the composite rows of Table 2 of the main paper into the five underlying metrics for the extraction arm and the matched distillation baseline.

Table 6. CIFAR-10 cifar_relu EQS detail

Metric	Extraction	Distillation	Gap
Metric 1 (in-dist fidelity)	56.40%	50.33%	+6.07 pt
Metric 2 (margin-cond., far bin)	60.12%	52.71%	+7.41 pt
Metric 3 (off-manifold, uniform)	34.20%	19.62%	+14.58 pt
Metric 4 (McNemar / CI)	$p \approx 1.2 \times 10^{-27}$		+6.07 pt [4.99, 7.18]
Metric 5 (structural cos / sign)	1.000 / 0.531	≈ 0 / —	$S = 0.710$ vs 0
EQS (structural)	53.6	30.2	+23.4

Table 7. CIFAR-10 cifar_leakyrelu EQS detail

Metric	Extraction	Distillation	Gap
Metric 1 (in-dist fidelity)	58.29%	52.86%	+5.43 pt
Metric 2 (margin-cond., far bin)	62.92%	54.86%	+8.06 pt
Metric 3 (off-manifold, uniform)	34.42%	21.16%	+13.26 pt
Metric 4 (McNemar / CI)	$p \approx 5.4 \times 10^{-22}$		+5.43 pt [4.36, 6.55]
Metric 5 (structural cos / sign)	1.000 / 0.537	≈ 0 / —	$S = 0.836$ vs 0
EQS (structural)	57.7	31.5	+26.2

The breakdown shows where the composite gap comes from. On both victims the off-manifold gap (Metric 3) is the largest functional gap by a wide margin, +14.58 pt on `cifar_relu` and +13.26 pt on `cifar_leakyrelu`, more than double the in-distribution fidelity gap of +6.07 pt and +5.43 pt. The two arms are close on the data manifold and far apart off it, which is the signature of structural recovery rather than fit. Metric 5 sharpens this. The extraction arm carries exact directions with structural score 0.710 and 0.836 while the distillation arm scores zero by construction, since its rows match no recovered direction. The McNemar test rules out seed noise as the source of the in-distribution gap, with $p \approx 1.2 \times 10^{-27}$ and $p \approx 5.4 \times 10^{-22}$, both far below any conventional threshold. The composite follows directly, an extraction EQS of 53.6 against 30.2 for `cifar_relu` and 57.7 against 31.5 for `cifar_leakyrelu`, gaps of +23.4 and +26.2.

7 Additive Reconstruction Ablation

Table 8 builds the Phase-3 reconstruction one component at a time on each `make_blobs` victim, recording held-out agreement and structural EQS at every stage. Stages are cumulative. The distillation baseline freezes nothing and is shown for contrast.

Table 8. Additive Phase-3 ablation on the six `make_blobs` victims, evaluated on the held-out slice `X_test3`. Each stage adds one component to the stage above it. Agreement (Agr) and structural EQS are reported per stage; the distillation baseline is non-staged.

Arch	Stage	ReLU		Leaky ReLU	
		Agr	EQS	Agr	EQS
tiniest	RAW (Phase 1+2)	12.40%	23.6	12.50%	30.9
	+ bias recovery	22.80%	28.9	2.80%	24.4
	+ output-layer fit	92.40%	66.8	98.00%	70.9
	+ sign search	99.40%	70.7	97.90%	70.8
	+ frozen refine (full)	99.70%	71.2	98.90%	71.5
	distillation baseline	96.00%	54.4	98.60%	57.1
tinier	RAW (Phase 1+2)	23.30%	32.5	25.00%	35.4
	+ bias recovery	0.00%	17.5	6.58%	25.9
	+ output-layer fit	99.94%	75.6	99.92%	74.7
	+ sign search	100.00%	79.6	100.00%	77.9
	+ frozen refine (full)	100.00%	81.3	100.00%	78.8
	distillation baseline	100.00%	60.7	100.00%	60.2
tiny	RAW (Phase 1+2)	3.00%	19.2	10.68%	26.7
	+ bias recovery	14.34%	24.8	12.60%	27.7
	+ output-layer fit	100.00%	71.0	100.00%	72.5
	+ sign search	100.00%	74.1	100.00%	75.7
	+ frozen refine (full)	100.00%	75.3	100.00%	77.1
	distillation baseline	100.00%	55.2	100.00%	55.3

The ablation separates the agreement story from the structural one. On every victim the output-layer regression carries almost the whole agreement lift, taking `tiny_relu` from 3.00% to 100.00% in a single fit, because the recovered hidden features are already close to the victim’s and a linear decoder on top of them is all the task needs. Once agreement saturates, the sign search and the frozen refinement add little or nothing to it, which is expected on tasks this separable. The structural EQS tells the part that agreement hides. Sign search lifts EQS on every victim after agreement has saturated, by up to four points, and frozen refinement adds a further point or two, because both are repairing structure that perfect agreement does not require but the scorecard rewards. The bias-recovery row reads as a regression on several victims, dropping agreement before the output layer exists, but it is a prerequisite rather than a contributor. Its estimates feed the prefix activations that the output-layer fit and the sign search depend on, and its effect is measured here against a still-random decoder, so the intermediate number is not meaningful in isolation. The contrast with the distillation baseline holds throughout. The full pipeline and the baseline tie on agreement for the separable victims, yet the full pipeline scores fifteen to twenty EQS points higher, the same structural advantage the two-arm comparison reports.

References

1. Maas, A. L., Hannun, A.Y., Ng, A.Y.: Rectifier nonlinearities improve neural network acoustic models. In: ICML Workshop on Deep Learning for Audio, Speech and Language Processing (2013)